Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Sztuczna inteligencja w grafice komputerowej

103A-INxxx-MSP-SIGK

Projekt 1

Modyfikacja obrazów

Prowadzący: dr inż. Michał Chwesiuk

Zespół nr 10 - Wściekliza

Członkowie zespołu: Karol Łukasik | Monika Woźniak

Numer albumu: 310801 | 347435

1. INFORMACJE I ZAŁOŻENIA DOT. REALIZACJI PROJEKTU

W ramach projektu zrealizowano wybrane zadania z zakresu rekonstrukcji i modyfikacji obrazów z użyciem frameworka **PyTorch**.

W ramach pracy nad projektem:

- Przeskalowano dane do wymaganego formatu 256x256 pikseli
- Zrealizowano wybrane dwa zadania (zwiększanie rozdzielczości oraz odszumianie)
- Zbadano jakość stworzonych rozwiązań za pomocą wskazanych metryk (SNE, PSNR, SSIM, LPIPS)

Kod do projektu znajduje się w naszym repozytorium na GitHub: https://github.com/kmlukasik/SIGK-26Z-AI-in-computer-graphics-Wscieklizna

2. ZBIORY DANYCH

2.1. Wybór zbiorów danych

Wykorzystane zbiory danych:

- https://data.vision.ee.ethz.ch/cvl/DIV2K/DIV2K_train_HR.zip
- https://data.vision.ee.ethz.ch/cvl/DIV2K/DIV2K_valid_HR.zip

Zawierają obrazy w wysokiej rozdzielczości, o różnych rozmiarach i orientacjach (pionowych i poziomych).

Charakterystyka wybranych zbiorów danych:

- **DIV2K_train_HR** – zbiór treningowy: 800 obrazów w 94 rozmiarach, najczęściej 2040x1356 (429 obrazów).
- **DIV2K_valid_HR** – w naszym projekcie zbiór testowy: 100 obrazów w 24 rozmiarach, najczęściej 2040x1356 (53 obrazy).

2.2. Przygotowanie zbiorów danych do realizacji projektu

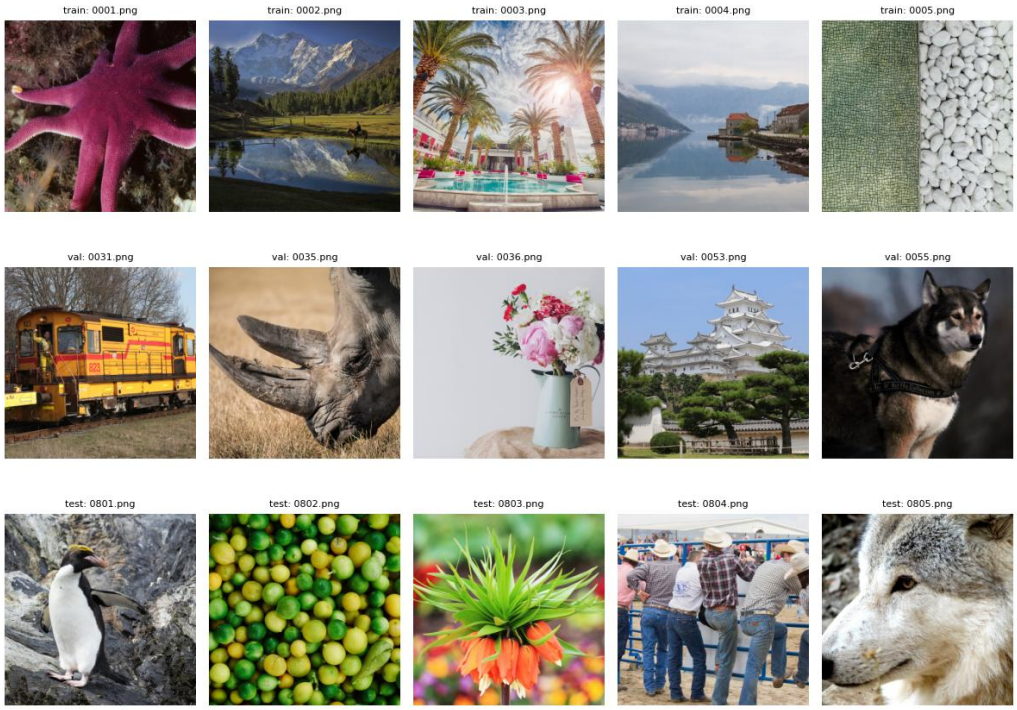
Zgodnie z założeniem projektu obrazy przeskalowano do rozdzielczości 256 × 256:

- Funkcja change_size(path) skaluje obraz tak, by jego krótszy bok miał 256 pikseli (z zachowaniem proporcji, interpolacja obszarowa INTER_AREA, zalecana przy zmniejszaniu obrazów).
- Z przeskalowanego obrazu wycinany jest środkowy fragment 256 × 256 pikseli (center crop), co oznacza, że brzegi obrazu są odrzucane.

Podział danych:

- **Zbiór treningowy (720 obrazów):** DIV2K_train_HR bez 80 losowo wybranych obrazów (random.seed(0), podział powtarzalny).

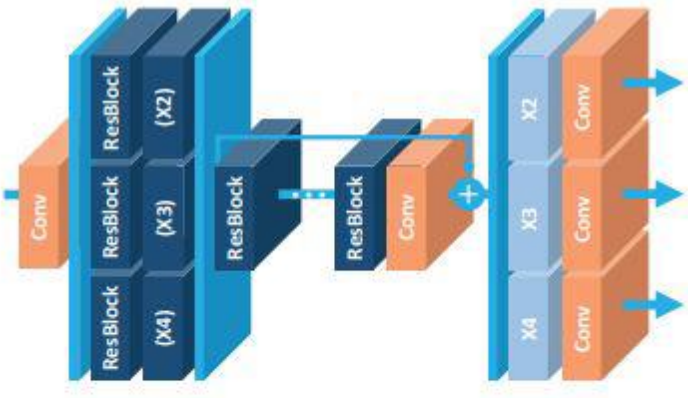
- **Zbiór walidacyjny (80 obrazów):** 80 losowo wybranych obrazów z DIV2K_train_HR; służy do kontroli uczenia, wyboru najlepszego modelu i doboru parametrów metody bazowej.
- **Zbiór testowy (100 obrazów):** cały DIV2K_valid_HR (oryginalny zbiór testowy DIV2K nie ma publicznych obrazów HR).



Rys. 1
Pięć pierwszych obrazów z każdego ze zbiorów (po podziale na treningowy, walidacyjny i testowy)

3. WYBRANE ARCHITEKTURY

3.1. Zwiększanie rozdzielczości



Rys. 2
Schemat architektury EDSR (32 bloki rezydualne, 256 kanałów), wejście w niskiej rozdzielczości (np. 3 × 64 × 64), wyjście 3 × 256 × 256.

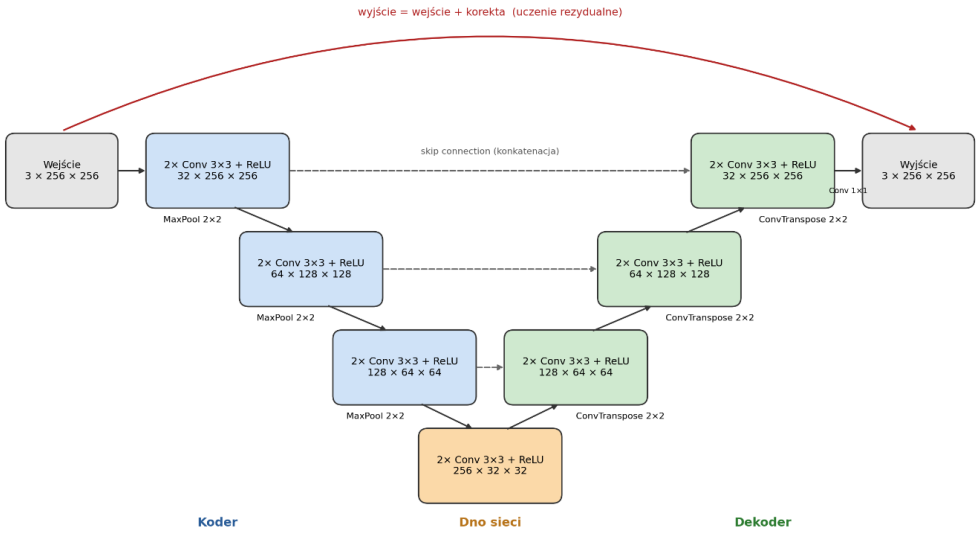
Zastosowano sieć EDSR (*Enhanced Deep Super-Resolution*). Przetwarzanie rozpoczyna się od pojedynczej warstwy splotowej, po której następuje głęboki moduł ekstrakcji cech złożony z 32 bloków rezydualnych (każdy posiada 256 kanałów). Każdy blok składa się z dwóch konwolucji 3×3 przedzielonych funkcją aktywacji ReLU. Zwiększenie wymiarów do docelowego formatu 256 × 256 następuje dopiero na samym końcu sieci za pomocą modułu upsamplingu (warstwy PixelShuffle). Końcowa konwolucja 3×3 zwraca trzy kanały RGB, a wyjściowy tensor jest twardo przycinany do przedziału przestrzeni barw $\langle 0.0, 1.0 \rangle$. Sieć jest bardzo pojemna i liczy około 43 milionów parametrów.

Wykorzystano globalne uczenie rezydualne – wyodrębnione cechy z początku sieci są dodawane do tych z końca modułu głównego (przed upsamplingiem), co ułatwia propagację informacji o pierwotnej strukturze obrazu.

Wybór architektury uzasadniają:

- Upsampling na końcu sieci: wykonywane większości operacji splotowych w niskej rozdzielczości zamiast na powiększonym wejściu, zmniejsza złożoność obliczeniową i przyspiesza działanie modelu
- Brak warstw normalizujących (BatchNorm) – ogranicza koszt pamięciowy.

3.2. Odszumianie



Rys. 3
Schemat architektury U-Net (base = 32), wejście i wyjście 3 × 256 × 256

Zastosowano sieć U-Net: koder trzykrotnie zmniejsza obraz (256 → 128 → 64 → 32) za pomocą max poolingu, zwiększając liczbę kanałów (32 → 64 → 128 → 256), a dekoder przywraca rozmiar za pomocą konwolucji transponowanych i łączy go z cechami kodera z tych samych poziomów (skip connections). Każdy blok to dwie konwolucje 3×3 z ReLU, a końcowa konwolucja 1×1 zwraca trzy kanały. Sieć ma 1 925 667 parametrów (Rys. 2).
Wyjściem sieci jest korekta dodawana do obrazu wejściowego (uczenie rezydualne). Wejście i wyjście mają rozmiar 3 × 256 × 256, więc ta sama architektura może służyć do odszumiania

oraz po wstępnym powiększeniu obrazu metodą bikubiczną do 256 × 256, do zwiększania rozdzielczości.

Wybór tej architektury uzasadniają:

- *Skip connections* - zachowują krawędzie i detale, a to jest główny problem bilaterala, który rozmywa teksturę (porównanie w rozdziale z wynikami).
- *Uczenie rezydualne* - wynik jest bliski wejściu, więc sieci łatwiej jest nauczyć się małej poprawki.

Dla prostoty zastosowano sieć bez warstw normalizujących (BatchNorm). Wpływu tej decyzji na wyniki nie badano.

4. METRYKI

Jakość oceniano metrykami opisanymi na wykładach:

- **SNE** *(Squared Norm Error – niższe lepsze)* – suma kwadratów różnic odpowiadających sobie pikseli; nie uwzględnia percepcji (lekkie rozmycie wygląda dobrze, ale daje wysokie SNE).
- **PSNR** *(Peak Signal-to-Noise Ratio – wyższe lepsze)* – stosunek maksymalnej wartości piksela do błędu średniokwadratowego, w decybelach; podobnie jak SNE słabo odzwierciedla jakość wizualną.
- **SSIM** *(Structural Similarity Index – wyższe lepsze)* – porównuje obrazy fragment po fragmencie pod względem jasności, kontrastu i struktury; wartości od 0 do 1, gdzie 1 oznacza obrazy identyczne.
- **LPIPS** *(Learned Perceptual Image Patch Similarity – niższe lepsze)* – odległość między cechami obrazów wyekstrahowanymi przez wyuczoną sieć CNN (AlexNet); odzwierciedla różnice dostrzegane przez człowieka.

5. REALIZACJA WYBRANYCH ZADAŃ

5.1. Zwiększanie rozdzielczości

Parametry danych wejściowych: obrazy w rozdzielczościach 32×32, 64×64.

Przygotowanie zbioru danych: obrazy o obniżonej rozdzielczości generowano dynamicznie w klasie UpscaleDataset za pomocą funkcji cv2.resize z interpolacją cv2.INTER_AREA. Obrazy wejściowe zmniejszano z formatu bazowego 256x256 bezpośrednio do wymiarów 64x64 oraz 32x32.

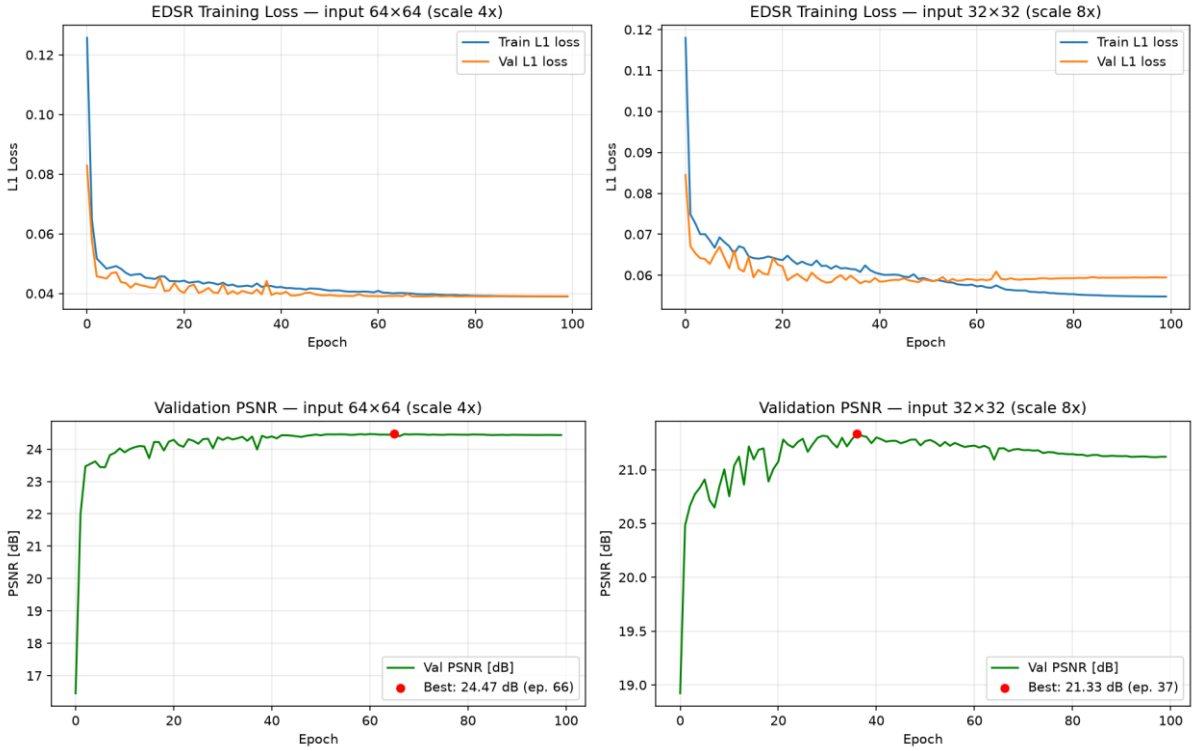


Rys. 4

Obrazy w formacie bazowym 256x256 oraz obrazy przeskalowane do rozdzielczości 64x64 px oraz 32x32 px..

Metoda bazowa do porównania wyników: interpolacja bikubiczna (resize z flagą cv2.INTER_CUBIC z biblioteki OpenCV), wywoływana przez funkcję bicubic_upscale z pliku baselines.py.

Proces uczenia się

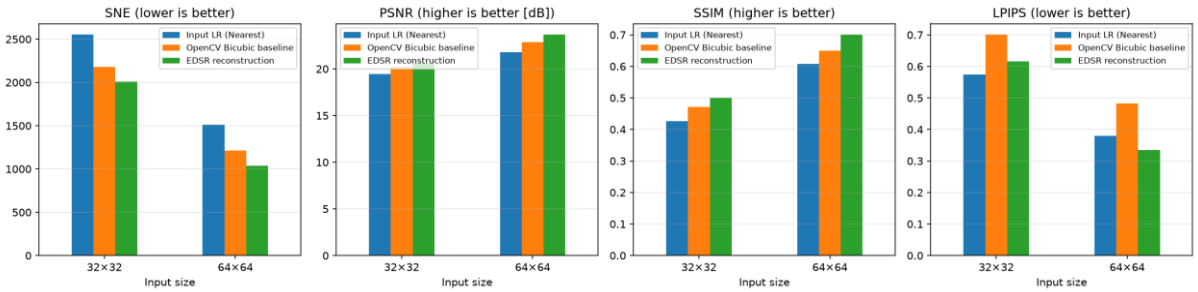


Rys. 5

Krzywe uczenia: strata treningowa (górny rząd) i PSNR na zbiorze walidacyjnym (dolny rząd) dla skal 4x (64->256) oraz 8x (32->256). Linia przerywana oznacza PSNR metody bazowej (bicubic), a czerwona kropka najlepszą epokę.

Dla każdego współczynnika skalowania wytrenowano osobną sieć EDSR (32 bloki rezydualne, 256 kanałów, skalowanie rezydualne 0.1, brak warstw normalizacji wsadowej) przez 100 epok: optymalizator Adam (początkowe tempo uczenia 10^{-4}), batch size 8, funkcja straty L_1. Dane wejściowe LR (64x64 lub 32x32) generowano dynamicznie w locie przy użyciu cv2.INTER_AREA. Po każdej epoce wyznaczano metryki na zbiorze walidacyjnym (80 obrazów) i zapisywano wagi modelu o najwyższym PSNR. Dla skali 4x (64->256) model uczył się bardzo stabilnie: strata walidacyjna osiągnęła minimum 0.0390, a walidacyjny PSNR wzrósł z początkowych 16.45 dB do maksimum 24.47 dB w 63. epoce, stabilizując się w fazie łagodnego nasycenia. Dla skali 8x (32->256) – z uwagi na drastyczną utratę informacji wejściowego obrazu – model osiągnął optimum nieco wcześniej: minimalną stratę walidacyjną 0.0580 oraz najwyższy PSNR na poziomie 21.33 dB w 37. epoce, po czym zaczął wykazywać cechy lekkiego przeuczenia przy dalszym, powolnym spadku błędu na zbiorze treningowym (na przestrzeni pozostałych kilkudziesięciu epok).

Wyniki na zbiorze testowym

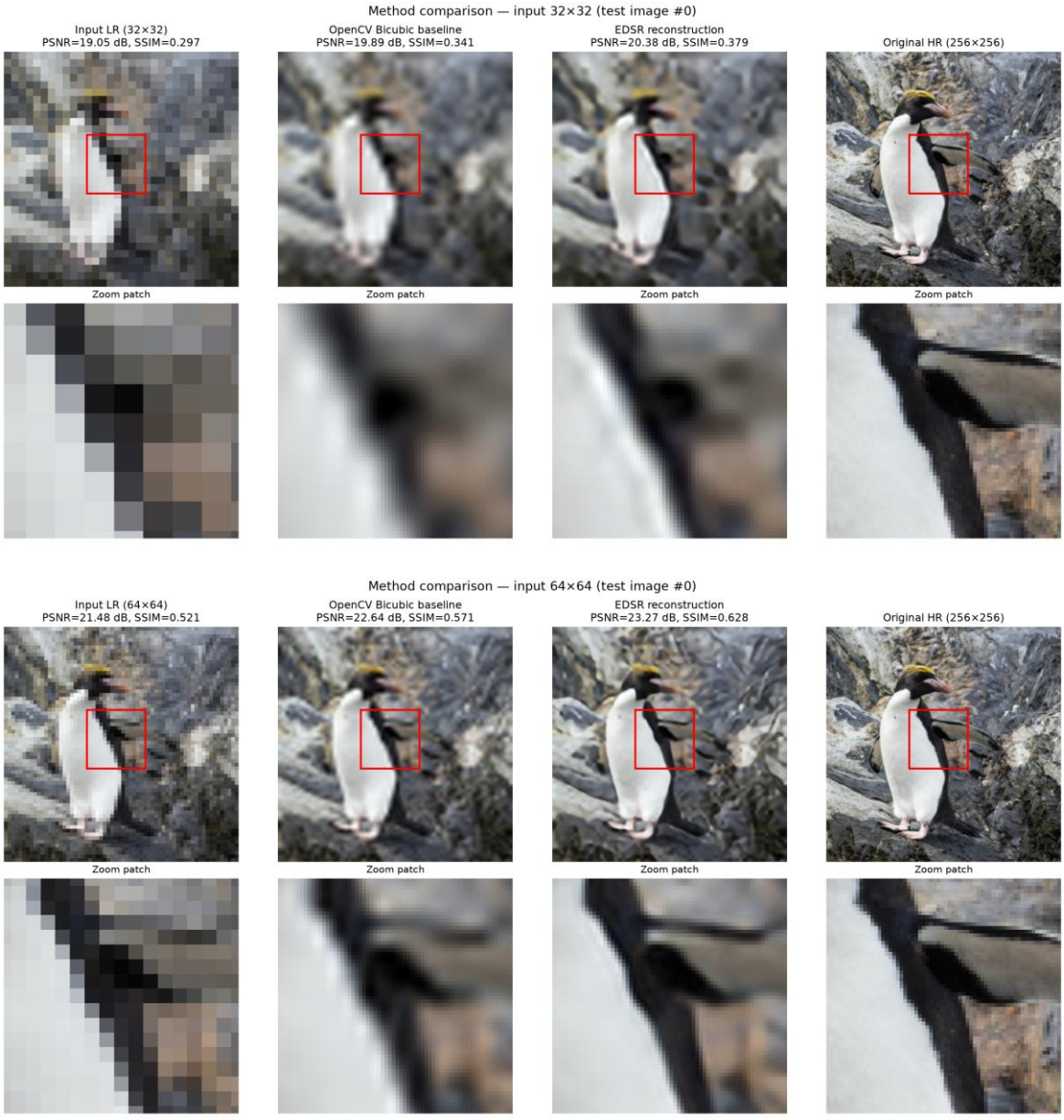


Rys. 6
Wykresy metryk

Tab. 1. Wyniki na zbiorze testowym (100 obrazów) dla wejść 64x64 oraz 32x32.

Rozmiar wejścia	Metoda	SNE	PSNR	SSIM	LPIPS
64×64	**Input LR (Nearest)**	1509.79	21.790	0.6080	0.3799
64×64	**OpenCV Bicubic baseline**	1212.76	22.878	0.6493	0.4827
64×64	**EDSR reconstruction**	1036.41	23.671	0.7005	0.3355
32×32	**Input LR (Nearest)**	2551.96	19.429	0.4264	0.5740
32×32	**OpenCV Bicubic baseline**	2178.77	20.196	0.4713	0.7007
32×32	**EDSR reconstruction**	2010.46	20.591	0.5002	0.6153

Tabela 1. przedstawia całościowe wyniki na niezależnym zbiorze testowym (100 obrazów). Dodano również metodę Nearest Neighbor (najbliższego sąsiada), aby zobrazować punkt odniesienia samej degradacji przestrzennej. Dla obu badanych skal model EDSR przewyższył klasyczną interpolację bikubiczną pod kątem wierności pikselowej (wyższe PSNR, niższe SNE) i strukturalnej (wyższe SSIM). W przypadku wejścia 64x64, PSNR wzrósł z 22.88 dB do 23.67 dB, a w trudniejszym zadaniu 32x32 z 20.20 dB do 20.59 dB. Co ciekawe, w przypadku metryki percepcyjnej LPIPS, metoda bazowa Bicubic osiągnęła wyniki gorsze nawet od surowego powiększenia Nearest (0.4827 vs 0.3799 dla skali 4x), co wynika z właściwości algorytmu bikubicznego, który silnie rozmywa tekstury – efekt ten jest surowo karany przez sieć VGG/AlexNet oceniającą LPIPS. Sieć EDSR skutecznie zrekonstruowała część krawędzi, osiągając we wszystkich przypadkach najlepsze (najniższe) wyniki LPIPS.



Rys. 7
Porównanie metod zwiększania rozdzielczości dla obrazu testowego (wejście 32×32 i 64×64)

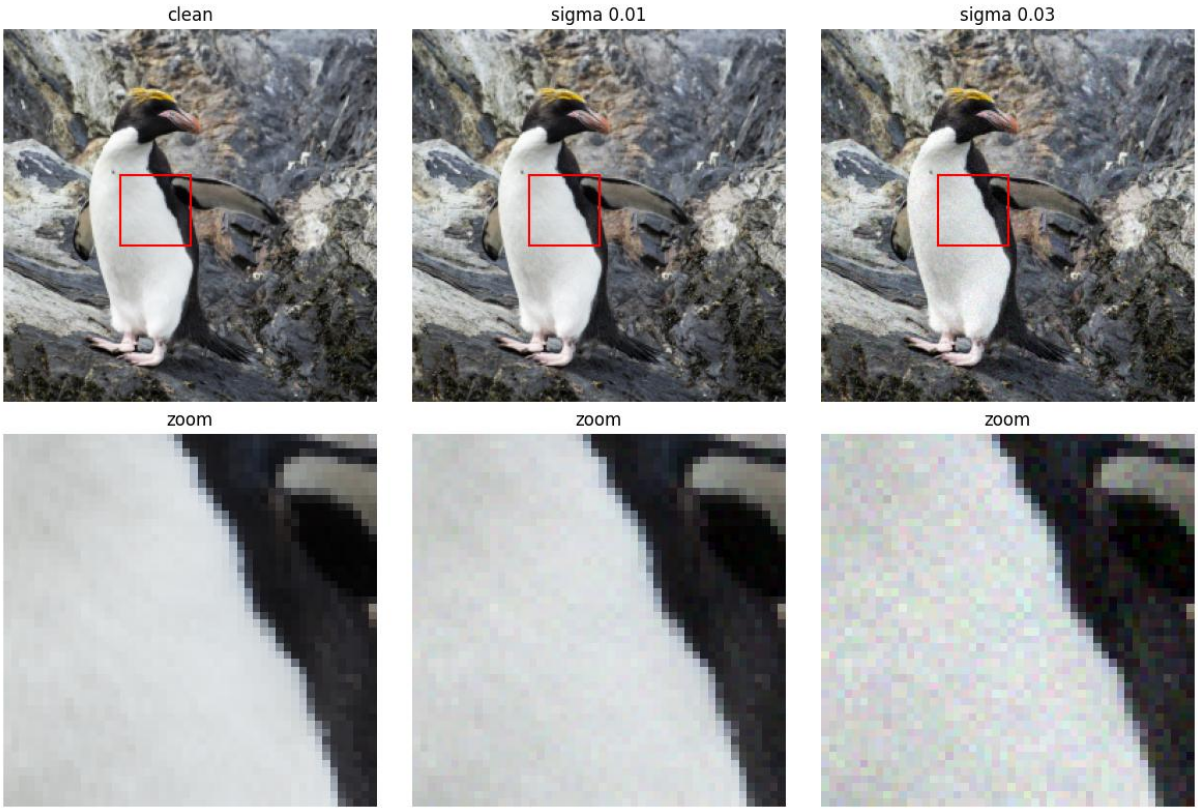
Wnioski

Sieć EDSR przewyższa klasyczny algorytm bikubiczny we wszystkich analizowanych metrykach, niezależnie od stopnia początkowej degradacji.

Degradacja obrazu 256x256 do formatu 32x32 powoduje bezpowrotną utratę ponad 98% pikseli. Poprawa metryk dla skali 8x, choć jest zauważalna (wzrost PSNR o 0,4dB), jest znacznie mniejsza niż dla skali 4x (wzrost o 0.8dB). Przy tak małej ilości danych wejściowych sieć ma problem z przewidywaniem poprawnej struktury.

5.2. Odszumianie

- **Parametry danych wejściowych:** współczynniki dla szumu gaussowskiego σ : 0.01,0.03 dla znormalizowanych obrazków o wartościach w przedziale < 0.0, 1.0 >
- **Przygotowanie zbioru danych:** szum gaussowski generowano własną funkcją add_gaussian_noise (NumPy): do obrazu z przedziału <0, 1> dodawano szum $N(0, \sigma^2)$, a wynik obcinano do <0, 1> (jest to równoważne skimage.util.random_noise(mode='gaussian', var=σ^2)). Dla zbioru testowego szum ma stałe ziarno (osobne dla każdego obrazu), dzięki czemu wyniki są powtarzalne.



Rys. 8
Obraz czysty oraz ten sam obraz zaszumiony dla σ = 0.01 i σ = 0.03 (zbiór testowy)

Dla σ = 0.01 szum jest ledwo widoczny, a dla σ = 0.03 wyraźnie widać ziarno na jednolitych powierzchniach (dolny rząd: powiększony fragment zaznaczony czerwonym kwadratem). Szum jest losowany niezależnie dla każdego kanału koloru, więc ma barwną strukturę.

Metoda bazowa: denoise_bilateral z biblioteki skimage, wywoływana przez funkcję bilateral_denoise z pliku baselines.py.

Parametry filtra (sigma_color, sigma_spatial) dobrano przeszukiwaniem siatki na 10 obrazach zbioru walidacyjnego, osobno dla każdego σ, według PSNR; zbioru testowego do doboru nie używano. Domyślne parametry dawały PSNR ok. 20 dB, czyli znacznie gorzej niż obraz zaszumiony.

Tab. 2. PSNR [dB] filtru bilateralnego dla σ = 0.01 w zależności od parametrów sigma_color i sigma_spatial (zbiór walidacyjny, 10 obrazów). Zielonym tłem zaznaczono najlepszy wynik.

	sigma_spatial		
sigma_color	**2**	**3**	**4**
0.003	41,24	41,37	41,41
0.005	41,21	41,52	**41,61**
0.01	39,08	39,47	39,53
0.02	35,36	35,56	35,49

Obraz zaszumiony (bez przetwarzania): 40,03 dB.

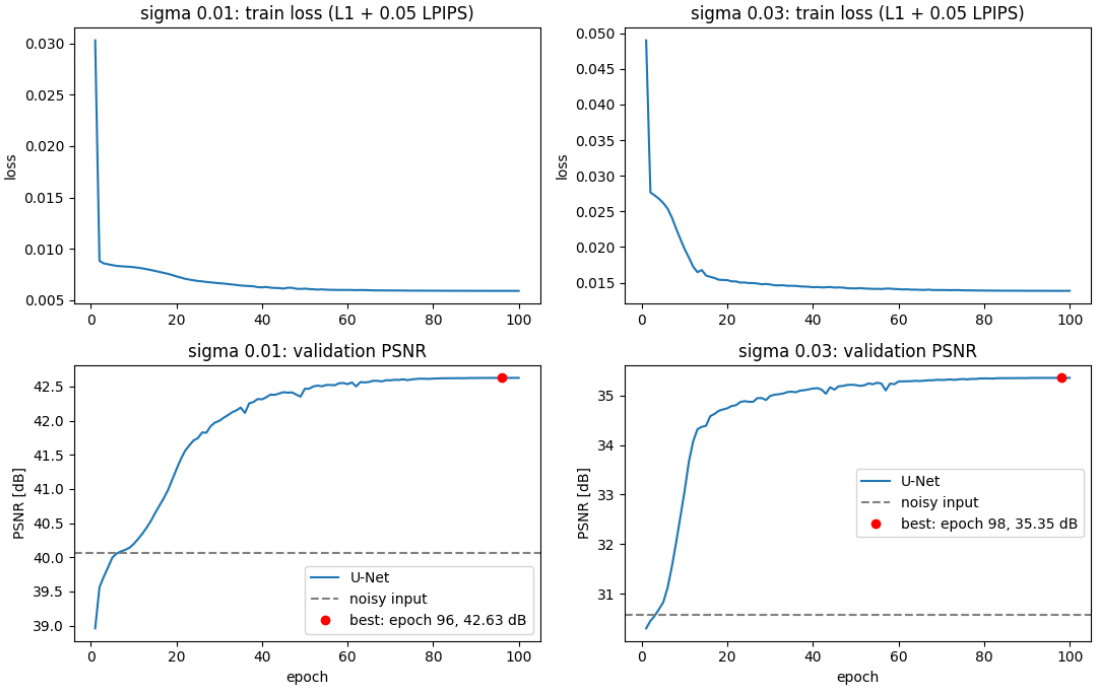
Tab. 3. PSNR [dB] filtru bilateralnego dla σ = 0.03 w zależności od parametrów sigma_color i sigma_spatial (zbiór walidacyjny, 10 obrazów). Zielonym tłem zaznaczono najlepszy wynik.

	sigma_spatial		
sigma_color	**2**	**3**	**4**
0.003	30,85	30,87	30,89
0.005	31,60	31,62	31,62
0.01	33,43	33,51	33,48
0.02	33,86	33,97	33,88

Obraz zaszumiony (bez przetwarzania): 30,53 dB.

Dla σ = 0.01 najlepszy wynik na zbiorze walidacyjnym dał zestaw sigma_color = 0.005, sigma_spatial = 4 (PSNR 41,61 dB wobec 40,03 dB dla obrazu zaszumionego), a dla σ = 0.03 zestaw sigma_color = 0.02, sigma_spatial = 3 (33,97 dB wobec 30,53 dB).
Zbyt duże sigma_color powodowało rozmywanie krawędzi i spadek PSNR poniżej poziomu obrazu zaszumionego, a zbyt małe uniemożliwiało skuteczne usunięcie szumu.

Proces uczenia się



Rys. 9. Krzywe uczenia: strata treningowa (górny rząd) i PSNR na zbiorze walidacyjnym (dolny rząd) dla σ = 0.01 i σ = 0.03. Linia przerywana oznacza PSNR obrazu zaszumionego, a czerwona kropka najlepszą epokę.

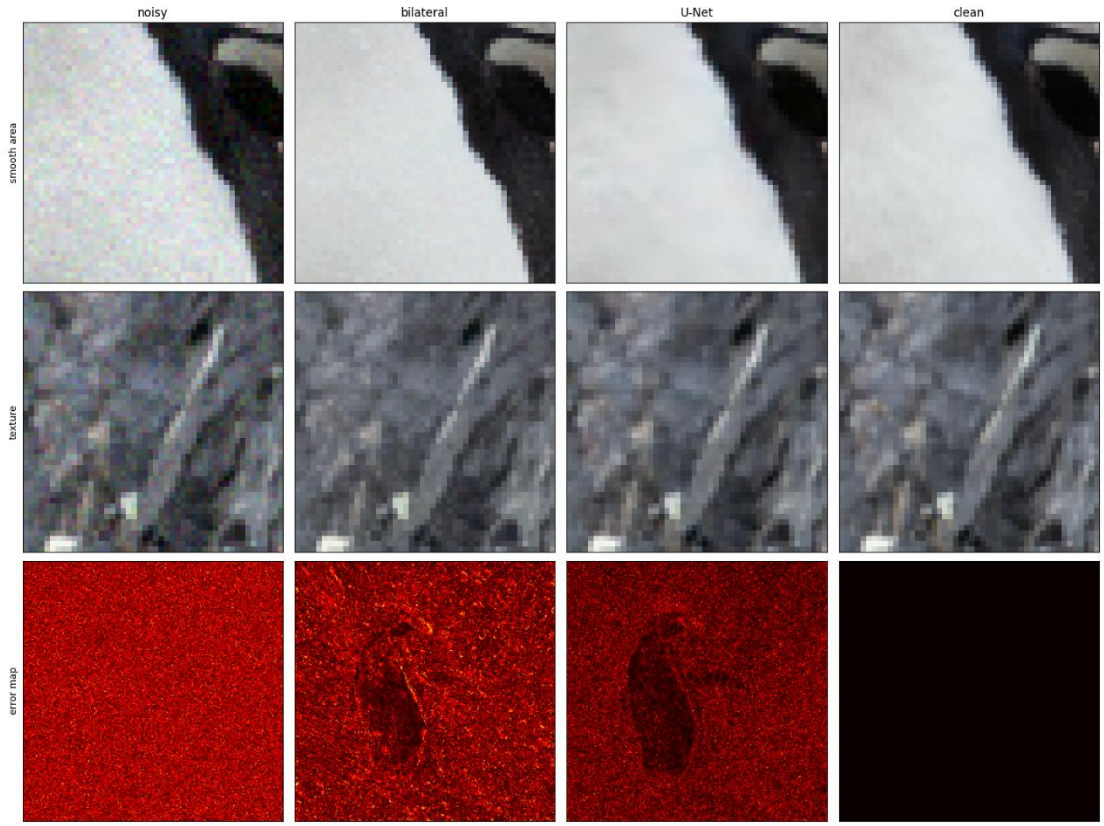
Dla każdej wartości σ wytrenowano osobny model U-Net przez 100 epok: optymalizator Adam (tempo uczenia $2 \cdot 10^{-4}$, harmonogram cosine), partia 16, funkcja straty L1 + 0,05·LPIPS. Szum na zbiorze treningowym losowano od nowa w każdej epoce, a po każdej epoce liczono PSNR na zbiorze walidacyjnym (80 obrazów) i zapisywano najlepszy model (Rys. 4). PSNR szybko rósł w pierwszych dziesiątkach epok, a od ok. 80. epoki prawie się ustabilizował (zgodnie z malejącym tempem uczenia). Najlepsze modele osiągnęły na walidacji 42,63 dB dla σ = 0.01 (epoka 96) i 35,35 dB dla σ = 0.03 (epoka 98), czyli ok. 2,6 i 4,8 dB więcej niż obraz zaszumiony.

Tabela 4 przedstawia wyniki na całym zbiorze testowym (100 obrazów); dla SNE i LPIPS niżej = lepiej, dla PSNR i SSIM wyżej = lepiej. Dla obu poziomów szumu U-Net uzyskał lepsze wyniki niż metoda bazowa we wszystkich metrykach. Dla σ = 0.01 PSNR wzrósł z 41,11 dB (bilateral) do 42,57 dB, a dla σ = 0.03 z 33,11 dB do 35,23 dB; SNE spadł odpowiednio z 15,78 do 11,24 i ze 100,18 do 62,16. Przy małym szumie obraz zaszumiony jest już bliski oryginałowi (SSIM 0,9706), więc możliwa poprawa jest mniejsza. Ponieważ LPIPS był składnikiem funkcji straty, porównanie w tej metryce jest dla sieci korzystniejsze i należy je interpretować ostrożnie.

Tab. 4. Wyniki na zbiorze testowym (100 obrazów) dla σ = 0.01 i σ = 0.03.

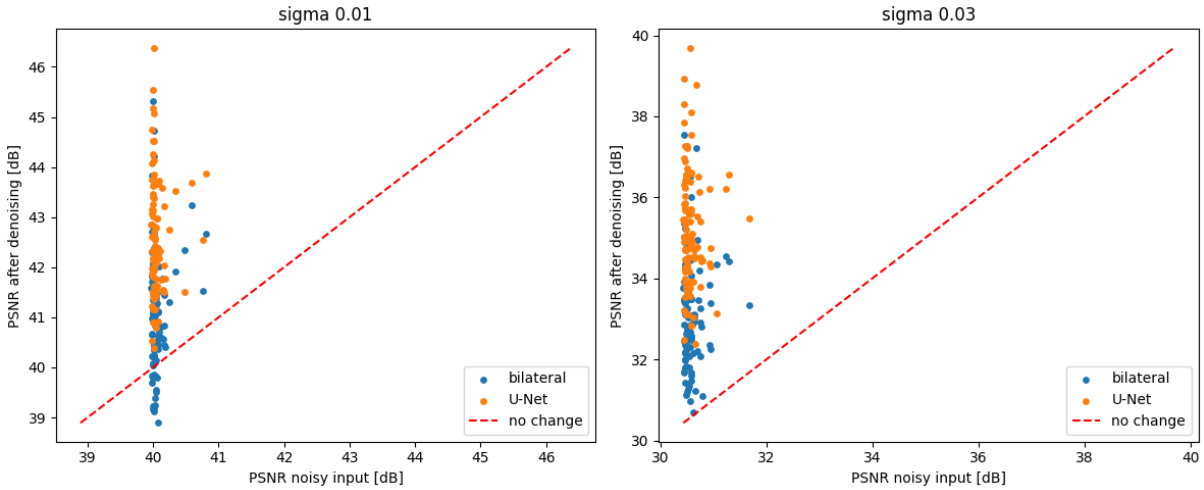
Metoda	SNE	PSNR	SSIM	LPIPS
noisy input sigma 0.01	19.401	40.0599	0.9706	0.0084
bilateral denoise sigma 0.01	15.7769	41.1054	0.986	0.0055
unet denoise sigma 0.01	11.2385	42.5731	0.9899	0.0021
noisy input sigma 0.03	171.915	30.587	0.827	0.0685
bilateral denoise sigma 0.03	100.1835	33.1078	0.9357	0.0411
unet denoise sigma 0.03	62.1605	35.23	0.9596	0.0139

Na zbliżeniach (Rys. 10) U-Net usuwa szum zarówno na gładkim obszarze, jak i na teksturze, zachowując więcej szczegółów niż filtr bilateralny, a jego mapa błędu jest ciemniejsza, zwłaszcza na korpusie pingwina i skałach w tle. Błędy obu metod koncentrują się na krawędziach obiektów.



Rys. 10
Porównanie wyników dla σ = 0.03: zbliżenia fragmentów (gładki obszar i tekstura) oraz mapa błędu względem obrazu czystego (jaśniej = większy błąd; skala ucięta do 0,1).

Rys. 11 pokazuje PSNR dla każdego z 100 obrazów testowych: U-Net osiąga zwykle wyższy PSNR niż filtr bilateralny. Dla σ = 0.01 bilateral pogorszył część obrazów względem obrazu zaszumionego, a na wykresie nie widać obrazów, które pogorszył U-Net.



Rys. 11
PSNR po odszumianiu w zależności od PSNR obrazu zaszumionego dla każdego z 100 obrazów testowych (σ = 0.01 po lewej, σ = 0.03 po prawej); punkty powyżej czerwonej linii oznaczają poprawę.

Wnioski. U-Net odszumia skuteczniej niż filtr bilateralny: poprawia wszystkie cztery metryki, a przewaga jest największa przy wyższym poziomie szumu ($\sigma = 0.03$). Filtr bilateralny dobrze wygładza jednolite obszary, ale rozmywa drobne tekstury, ponieważ nie odróżnia ich od szumu; sieć uczy się tej różnicy z danych. Wyniki pochodzą z jednego przebiegu treningu i jednej konfiguracji (bez badania rozrzutu), a parametry bilaterala dobrano na 10 obrazach, więc różnice rzędu setnych części dB należy traktować ostrożnie.